Lazy & IO

Lazy/IO · Defer expensive report until edge · Build with Waqar

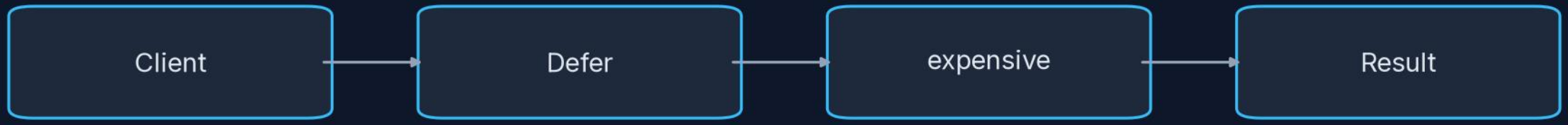
Today's Hook

- Problem: Defer expensive report until edge
- Pattern: Lazy/IO — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Defer expensive report until edge
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Lazy/IO: Defer expensive report until edge

Real-World Scenario

- Running example: Defer expensive report until edge
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.lazy.problem;

import java.util.*;

public final class NaiveReport {
  public static String build(){ System.out.println("BUILD"); return "report"; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.functional.lazy.problem;

import java.util.*;
import java.util.function.*;

final class Lazy<A> {
  private final Supplier<A> thunk; private A value; private boolean ready;
  Lazy(Supplier<A> thunk){ this.thunk=thunk; }
  static <A> Lazy<A> of(Supplier<A> s){ return new Lazy<>(s); }
  synchronized A get(){ if(!ready){ value=thunk.get(); ready=true; } return value; }
}
final class IO<A> {
  private final Supplier<A> effect; IO(Supplier<A> effect){ this.effect=effect; }
  static <A> IO<A> delay(Supplier<A> s){ return new IO<>(s); }
  <B> IO<B> map(Function<A,B> f){ return new IO<>(() -> f.apply(effect.get())); }
  A unsafeRun(){ return effect.get(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.lazy.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Lazy/IO: defer expensive report ===");
    System.out.println("--- naive ---"); System.out.println(NaiveReport.build());
System.out.println("--- lazy ---");
var lazy=Lazy.of(() -> { System.out.println("BUILD"); return "report"; });
System.out.println("created"); System.out.println(lazy.get()); System.out.println(lazy.get());
System.out.println("--- io ---");
System.out.println(IO.delay(() -> "edge").map(s -> s+"!").unsafeRun());
  }
}
```

Walkthrough Output

```
=== Lazy/IO: defer expensive report ===
--- naive ---
BUILD
report
--- lazy ---
created
BUILD
report
report
--- io ---
edge!
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Lazy/IO.

Related Patterns

- Contrast Lazy/IO with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Lazy/IO → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.